

JAVASCRIPT

Condicionais, repetições e coleções

Tomando decisões, repetindo tarefas e organizando dados em JavaScript.

```
> if (turma.pronta) console.log('Vamos!')  
Vamos!
```

Tarik Ponciano · Senac

VISÃO GERAL

O plano de hoje

- 01 Abertura e retomada do conteúdo anterior
- 02 Condicionais: if, else e switch
Pausa
- 03 Repetições: for, while e do...while
Pausa
- 04 Coleções: arrays e objetos
- 05 Mini-projeto integrador: inscrições de evento
- 06 Fechamento

OBJETIVOS

Ao final da aula você vai conseguir

Decidir

com **if**, **else if**, **else** e **switch**.

Repetir

tarefas com **for**, **while** e **do...while**.

Organizar

dados em arrays e objetos.

Transformar

coleções com **map**, **filter** e **reduce**.

E juntar tudo em um sistema de inscrições completo, com relatório final.

RETOMADA

O que já vimos antes

Sintaxe

const/let, tipos, **typeof**.

Exibição

console.log, template literals, formatação.

Coleta

@inquirer/prompts: input, number, select, confirm, checkbox.

```
const nome = await input({ message: 'Nome?' });
const idade = await number({ message: 'Idade?', required: true });
console.log(`${nome} tem ${idade} anos`); // string, number, template literal
```

NIVELAMENTO

Quem já usou if e for em outra linguagem?

A lógica é quase sempre a mesma. Muda a sintaxe.

CONDICIONAIS

if e else

```
if (idade ≥ 18) {  
  console.log('Maior de idade');  
} else {  
  console.log('Menor de idade');  
}
```

A condição fica entre **parênteses**.

O bloco fica entre **chaves**.

Sempre use chaves, mesmo com uma linha.

CONDICIONAIS

else if: várias faixas

```
if (nota ≥ 9) {  
  console.log('Conceito A');  
} else if (nota ≥ 7) {  
  console.log('Conceito B');  
} else if (nota ≥ 5) {  
  console.log('Conceito C');  
} else {  
  console.log('Conceito D');  
}
```

| Testa de cima para baixo.

| Para no **primeiro** verdadeiro.

| A ordem das condições importa.

CONDICIONAIS

Blocos { } criam escopo

```
if (true) {  
  let mensagem = 'só existe aqui dentro';  
  console.log(mensagem);  
}  
console.log(mensagem); // ReferenceError: mensagem is not defined
```

| {} cria um novo **escopo**.

| A variável não existe fora do bloco.

| **var** ignora essa regra: evite.

CONDICIONAIS

if com valores truthy/falsy

```
const nome = await input({ message: 'Apelido?' });
if (nome) {
  console.log(`Oi, ${nome}!`);
} else {
  console.log('Você não digitou nada.');
```



```
}
const carrinho = [];
if (carrinho.length) {
  console.log('Tem item no carrinho');
} else {
  console.log('Carrinho vazio');
}
```

O JS converte o valor para **boolean**.

if (lista.length) = "tem itens?"

Cuidado: **0** também é falsy.

CONDICIONAIS

switch: várias opções do mesmo valor

```
switch (diaSemana) {  
  case 0:  
  case 6:  
    console.log('Fim de semana');  
    break;  
  case 1: case 2: case 3: case 4: case 5:  
    console.log('Dia útil');  
    break;  
  default:  
    console.log('Dia inválido');  
}
```

Compara **um valor** com várias opções.

Sem **break**, cai para o próximo (fallthrough).

default é o caso restante.

CONDICIONAIS

Relembrando o ternário

```
// já visto na aula 1: se o if/else só ESCOLHE UM VALOR, o ternário resume tudo
let status;
if (idade ≥ 18) {
  status = 'liberado';
} else {
  status = 'bloqueado';
}

const status2 = idade ≥ 18 ? 'liberado' : 'bloqueado'; // mesma coisa
```

Se/então/senão que **devolve um valor**.

Bom para 2 opções, uma linha.

Não aninhe ternários.

CONDICIONAIS

Erros clássicos

```
let x = 5;
if ((x = 10)) {           // = atribui, não compara! isto sempre entra
  console.log('x agora vale', x);
}

switch (2) {
  case 1: console.log('um');
  case 2: console.log('dois'); // sem break: cai para o de baixo também
  case 3: console.log('três');
}
```

| **if (x = 10)** atribui, não compara.

| Use sempre **===**.

| Sem **break**, o switch "vaza".

ATIVIDADE 1 · SÓ INDIQUE A RESPOSTA

Preveja o resultado

```
let x = 7;  
if (x % 2 === 0) console.log('par'); else console.log('ímpar');  
  
let nome = '';  
if (nome) console.log('tem nome'); else console.log('sem nome');  
  
const a = 5, b = 5;  
if (a === b) console.log('iguais'); else console.log('diferentes');  
  
let n = 12;  
if (n > 10 && n < 20) console.log('entre 10 e 20');
```

1. Para cada bloco, escreva o que será impresso
2. Confira com a turma (ou teste no REPL)
3. Explique o porquê de cada resposta

Reescreva com switch

```
const tamanho = 2;  
if (tamanho === 1) console.log('P');  
else if (tamanho === 2) console.log('M');  
else if (tamanho === 3) console.log('G');  
else console.log('tamanho inválido');  
  
// Reescreva com switch (tamanho) { ... }
```

1. Reproduza o código acima em um arquivo novo e rode
2. Reescreva a mesma lógica usando **switch (tamanho) { ... }**
3. Rode as duas versões com **tamanho = 1, 2, 3 e 9** e compare

Desafio: e se **tamanho** fosse um texto ('P', 'M', 'G')? Ajuste as duas versões.

EXERCÍCIO 1 · REPRODUZA NO COMPUTADOR E TESTE

Boletim: nota → conceito

Nota	Conceito
nota $\geq$ 9	A
nota $\geq$ 7	B
nota $\geq$ 5	C
caso contrário	D

1. Peça a nota final do aluno (**number**, de 0 a 10, com decimais)
2. Classifique com **if/else if/else**, seguindo a tabela acima
3. Mostre: "Nota 8.5 -> Conceito B"

Desafio: refaça a classificação usando **switch (true) { case nota >= 9: ... }**.

Central de Suporte do Senac

O Senac abriu uma central de atendimento para dúvidas técnicas (internet, impressoras, sistema acadêmico...). Vocês vão construir o sistema que direciona cada chamado para o guichê certo.

1. Pergunte: a pessoa é aluno(a) matriculado(a)? (**confirm**)
2. Pergunte a categoria do problema (**select**: Internet, Impressora, Sistema Acadêmico, Outro)
3. Pergunte a urgência (**select**: Baixa, Média, Alta)

Regras de roteamento

Implemente estas regras, NESTA ordem de prioridade (a primeira que se aplicar, vale):

1. **1.** Sem matrícula confirmada → sempre "Recepção — verificar cadastro", não importa o resto
2. **2.** Urgência Alta **e** categoria Sistema Acadêmico → "Suporte Nível 2 (urgente)"
3. **3.** Urgência Alta (qualquer outra categoria) → "Guichê 1 (prioritário)"
4. **4.** Nos demais casos, direcione por categoria com **switch**: Internet → Guichê 2, Impressora → Guichê 3, Sistema Acadêmico → Guichê 4, Outro → Guichê Geral
5. **5.** Mostre também um tempo estimado de espera: Alta = 5 min, Média = 10 min, Baixa = 20 min

INTERVALO

Pausa

Na volta: repetições (for, while)

REPETIÇÕES

for: quando se sabe quantas vezes

```
for (let i = 1; i ≤ 5; i++) {  
  console.log('for', i);  
}  
// início; condição; incremento – roda enquanto a condição for true
```

3 partes: **início; condição; incremento.**

Roda enquanto a condição for **true**.

Ideal quando já se sabe o total de voltas.

REPETIÇÕES

while: enquanto for verdade

```
let contador = 3;  
while (contador > 0) {  
  console.log('while', contador);  
  contador--;  
}
```

Repete **enquanto** for **true**.

Sem partes fixas como o **for**.

Bom quando não se sabe o total de voltas.

REPETIÇÕES

do...while: roda ao menos uma vez

```
let n = 10;  
do {  
  console.log('isto roda uma vez, mesmo com n = 10');  
} while (n < 5);  
// testa a condição DEPOIS de rodar: garante ao menos uma execução
```

| Testa a condição **depois** de rodar.

| Garante ao menos **uma** execução.

| Bom para "peça e valide".

REPETIÇÕES

break e continue

```
for (let i = 1; i ≤ 10; i++) {  
  if (i === 4) break;           // para o laço imediatamente  
  console.log(i);  
}  
  
for (let i = 1; i ≤ 6; i++) {  
  if (i % 2 === 0) continue; // pula para a próxima volta  
  console.log(i);  
}
```

break: sai do laço **agora**.

continue: pula **esta** volta.

Funcionam em **for**,
while, **do...while**.

REPETIÇÕES

for...of: percorre valores

```
const frutas = ['maçã', 'banana', 'uva'];  
for (const fruta of frutas) {  
  console.log(fruta);  
}  
for (const letra of 'ABC') {  
  console.log(letra);  
}
```

Percorre os **valores** direto.

Funciona em arrays e strings.

É o laço mais usado com coleções.

REPETIÇÕES

for...in: percorre chaves

```
const aluno = { nome: 'Ana', idade: 20 };  
for (const chave in aluno) {  
  console.log(chave, '→', aluno[chave]);  
}  
// for...in é para as CHAVES de um objeto. Em array, evite: dá o índice como texto.
```

Percorre as **chaves** de um objeto.

Evite em arrays: prefira **for...of**.

Voltaremos a isto no bloco de objetos.

REPETIÇÕES

O padrão sentinela: repetir até parar

```
import { input } from '@inquirer/prompts';

let total = 0;
while (true) {
  const texto = await input({ message: 'Valor? (Enter em branco encerra)' });
  if (texto.trim() === '') break; // condição de parada
  total += Number(texto.replace(',', '.'));
}
console.log('Total:', total.toFixed(2));
```

while (true) roda até um **break**.

A condição de parada é um **if** por dentro.

Combina com os prompts vistos antes.

REPETIÇÕES

Erros clássicos

```
// off-by-one: errar o ≤ por <
for (let i = 0; i < 5; i++) {} // 5 voltas (0 a 4)
for (let i = 0; i ≤ 5; i++) {} // 6 voltas (0 a 5)

// esquecer de atualizar a variável: LAÇO INFINITO
let i = 0;
while (i < 5) {
  console.log(i);
  // faltou i++ aqui!
}
```

< × ≤ muda o total de voltas.

Sem atualizar a variável:
laço infinito.

Ctrl+C interrompe o
programa travado.

ATIVIDADE 3 · SÓ INDIQUE A RESPOSTA

Preveja o resultado

```
for (let i = 0; i < 3; i++) console.log(i);
```

```
let i = 5;
```

```
while (i > 0) { console.log(i); i -= 2; }
```

```
for (let i = 1; i ≤ 10; i++) {
```

```
  if (i % 4 === 0) continue;
```

```
  if (i > 7) break;
```

```
  console.log(i);
```

```
}
```

1. Para cada trecho, escreva os valores que serão impressos, em ordem
2. Confira com a turma (ou teste no REPL)
3. No terceiro trecho, explique por que alguns números não aparecem

Tabuada

1. Pergunte um número com **number()**
2. Use um **for** para mostrar a tabuada desse número, de 1 a 10
3. Formato: "7 x 1 = 7", "7 x 2 = 14", ...

Desafio: peça também o limite final da tabuada (em vez de fixar em 10).

Repetições na prática

1. **(a)** FizzBuzz: para 1 a 30, mostre "Fizz" (múltiplo de 3), "Buzz" (múltiplo de 5), "FizzBuzz" (múltiplo dos dois) ou o número
2. **(b)** Some todos os múltiplos de 3 ou de 5 entre 1 e 99
3. **(c)** Verifique se um número é primo (só divisível por 1 e por ele mesmo)

Desafio: no primo, pare de testar assim que achar um divisor (break) — não precisa continuar.

ATIVIDADE EM DUPLA · TURNO DE CAIXA

Cantina do Senac

É hora do fechamento de caixa da cantina. Vocês vão programar o sistema que registra as vendas de um turno, uma de cada vez, até o operador encerrar.

1. Repita (padrão sentinela): pergunte o **item vendido (input)**; Enter em branco encerra o turno
2. Para cada item, pergunte o **preço unitário** e a **quantidade (number)**

Regras do turno

Para cada venda, calcule o valor (preço × quantidade) e aplique:

1. **Desconto:** 5 unidades ou mais do MESMO item → 10% de desconto NAQUELE item
2. Vá somando: **total do turno, quantidade de vendas e quantas tiveram desconto**
3. Ao encerrar (Enter em branco), mostre o resumo do turno
4. **Meta:** se o total do turno for R\$ 100 ou mais, mostre uma mensagem de "meta batida"; senão, mostre quanto falta

INTERVALO

Pausa

Na volta: coleções (arrays e objetos)

Criar e acessar

```
const numeros = [5, 3, 8, 1, 9];  
console.log(numeros[0], numeros.length); // 5 5  
console.log(numeros.at(-1)); // 9 (último)  
numeros[1] = 30; // altera pelo índice  
console.log(numeros);
```

Índices começam em **0**.

length conta os itens.

at(-1) pega o último.

Adicionar e remover

```
const fila = ['Ana'];  
fila.push('Bia');           // adiciona no FIM  
fila.unshift('Caio');      // adiciona no INÍCIO  
console.log(fila);         // ['Caio', 'Ana', 'Bia']  
  
fila.pop();                // remove do FIM (e devolve o valor)  
fila.shift();              // remove do INÍCIO  
console.log(fila);         // ['Ana']
```

| **push/pop**: no fim.

| **unshift/shift**: no início.

| Todos alteram o array original.

Percorrer: for × for...of

```
const numeros = [10, 20, 30];

for (let i = 0; i < numeros.length; i++) {
  console.log(i, numeros[i]);    // usa o índice
}

for (const n of numeros) {
  console.log(n);                // usa o valor direto
}
```

| **for**: acesso ao **índice**.

| **for...of**: acesso ao **valor**.

| Prefira **for...of** quando possível.

Buscar dentro do array

```
const numeros = [5, 30, 8, 1, 9];  
numeros.includes(8);           // true  
numeros.indexOf(8);            // 2  
numeros.find((n) => n > 20);    // 30 (primeiro que passa)  
numeros.some((n) => n > 20);    // true (existe algum?)  
numeros.every((n) => n > 0);    // true (todos passam?)
```

includes/indexOf: valor exato.

find: primeiro que passa no teste.

some/every: existe algum?
/ todos?

map e filter

```
const numeros = [3, 7, 1, 9];

numeros.map((n) => n + 1);      // [4, 8, 2, 10] – transforma cada item
numeros.filter((n) => n > 5);    // [7, 9] – mantém quem passa no teste
// os dois SEMPRE devolvem um array NOVO; o original não muda
```

map: transforma
(mesmo tamanho).

filter: seleciona (tamanho
pode mudar).

Sempre devolvem um array **novo**.

reduce: junta tudo em um valor

```
[10, 20, 30].reduce((acumulado, atual) ⇒ acumulado + atual, 0);
```

```
// passo a passo:
```

```
// acumulado=0 , atual=10 → devolve 10
```

```
// acumulado=10 , atual=20 → devolve 30
```

```
// acumulado=30 , atual=30 → devolve 60
```

```
// resultado final: 60
```

Acumula um resultado a cada volta.

O 2º argumento é o **valor inicial**.

Bom para somas, totais e contagens.

sort: cuidado com a pegadinha

```
[10, 9, 1].sort();           // [1, 10, 9]   compara como TEXTO!  
[10, 9, 1].sort((a, b) => a - b); // [1, 9, 10] comparador numérico  
// sort() ALTERA o array original!  
// copie antes: [...arr].sort(...)
```

Sem comparador, ordena
como **texto**.

Números: **(a, b) => a - b**.

sort altera o array original.

Desestruturação e espalhamento

```
const [primeiro, segundo, ...resto] = [1, 2, 3, 4, 5];  
console.log(primeiro, segundo, resto);    // 1 2 [3, 4, 5]  
  
const copia = [...resto, 999];            // copia + acrescenta, sem alterar "resto"  
console.log(copia, resto);
```

const [a, b, ...resto] = arr

[...arr, novoItem] copia
+ acrescenta

Nunca altera o array original.

ATIVIDADE 5 · SÓ INDIQUE A RESPOSTA

Preveja o resultado

```
const n = [3, 7, 1, 9];  
console.log(n.map((x) => x + 1));  
console.log(n.filter((x) => x > 5));  
console.log(n.reduce((s, x) => s + x, 0));  
console.log([...n].sort((a, b) => a - b));
```

1. Para cada **console.log**, escreva o array (ou número) que aparece
2. Confira com a turma (ou teste no REPL)
3. No **sort**, explique por que o resultado NÃO seria o mesmo sem o comparador

Filtre e some

```
const precos = [45, 120, 30, 89, 15];  
// filtre os preços acima de 50 e depois some o total com reduce
```

1. Reproduza o array de preços acima
2. Use **filter** para pegar só os preços acima de R\$ 50
3. Use **reduce** para somar o total dos preços filtrados

Desafio: faça tudo em uma linha só, encadeando **.filter(...).reduce(...)**.

EXERCÍCIO 3 · REPRODUZA NO COMPUTADOR E TESTE

Notas da turma

```
const notas = [7.5, 4.0, 9.0, 6.5, 10, 3.5, 8.0];
```

1. Calcule a **média** da turma (reduce), com 1 casa decimal
2. Ache a **maior** e a **menor** nota (**Math.max/Math.min** com espalhamento)
3. Conte quantos foram **aprovados** (nota ≥ 7) com **filter**
4. Liste as notas aprovadas, em ordem crescente

Desafio: calcule o percentual de aprovação da turma.

Criar um objeto

```
const aluno = {  
  nome: 'Ana',  
  idade: 20,  
  notas: [8, 9, 7],  
};  
console.log(aluno.nome, aluno.notas.length);
```

Pares **chave: valor**, entre `{}`.

Cada valor pode ser de qualquer tipo.

Use quando os dados têm papéis diferentes.

Acessar, alterar, remover

```
console.log(aluno.nome);           // ponto
console.log(aluno['idade']);        // colchetes (para chave dinâmica)

aluno.idade = 21;                   // altera
aluno.email = 'ana@ex.com';        // cria, se não existia
delete aluno.email;                 // remove
```

Ponto ou **colchetes** para acessar.

Atribuir numa chave nova **cria** a propriedade.

delete remove uma propriedade.

Percorrer um objeto

```
const curso = { nome: 'DSM', cargaHoraria: 1200, vagas: 30 };

Object.keys(curso);      // ['nome', 'cargaHoraria', 'vagas']
Object.values(curso);    // ['DSM', 1200, 30]
Object.entries(curso);    // [['nome','DSM'], ['cargaHoraria',1200], ['vagas',30]]

for (const [chave, valor] of Object.entries(curso)) {
  console.log(chave, '→', valor);
}
```

Object.keys/values/entries

entries devolve pares
[chave, valor].

Combine com **for...of**.

Desestruturação e espalhamento

```
const { nome, ...resto } = aluno;  
console.log(nome, resto);           // Ana { idade: 21, notas: [...] }  
  
const { idade: idadeDoAluno = 0 } = aluno;    // renomeando + valor padrão  
console.log(idadeDoAluno);  
  
const atualizado = { ...aluno, idade: 22 };    // cópia com alteração  
console.log(atualizado.idade, aluno.idade);    // 22 21
```

const { nome } = objeto

Renomear: **{ idade: idadeDoAluno
}**

{ ...objeto, campo: novoValor }
cópia + altera

COLEÇÕES

O padrão mais usado: array de objetos

```
const produtos = [  
  { nome: 'Caderno', preco: 18.9, categoria: 'papelaria' },  
  { nome: 'Mochila', preco: 129.9, categoria: 'acessórios' },  
];  
  
for (const { nome, preco } of produtos) {  
  console.log(`${nome}: R$ ${preco.toFixed(2)}`);  
}  
  
produtos.filter((p) => p.categoria === 'papelaria');  
produtos.map((p) => p.nome.toUpperCase());  
produtos.reduce((total, p) => total + p.preco, 0);
```

É como uma **tabela** guardada na memória: cada objeto é uma linha, cada propriedade é uma coluna.

JSON: texto ↔ objeto

```
const texto = JSON.stringify(produtos); // array de objetos → texto
console.log(texto);

const devolta = JSON.parse(texto); // texto → array de objetos de novo
console.log(devolta[0].nome);
```

stringify: objeto → texto.

parse: texto → objeto.

É assim que dados viajam entre sistemas.

Objeto e Object.entries

1. Crie um objeto **curso** com **nome**, **cargaHoraria** e **vagas**
2. Percorra com **for (const [chave, valor] of Object.entries(curso))**
3. Mostre cada par como "chave: valor"

Desafio: crie uma cópia do objeto com **vagas** reduzida em 1, sem alterar o original.

EXERCÍCIO 4 · REPRODUZA NO COMPUTADOR E TESTE

Catálogo de produtos

```
const produtos = [  
  { nome: 'Caderno', preco: 18.9, categoria: 'papelaria', estoque: 40 },  
  { nome: 'Mochila', preco: 129.9, categoria: 'acessórios', estoque: 8 },  
  { nome: 'Caneta', preco: 3.5, categoria: 'papelaria', estoque: 120 },  
  { nome: 'Fone de ouvido', preco: 79.9, categoria: 'eletrônicos', estoque: 15 },  
  { nome: 'Garrafa', preco: 45.0, categoria: 'acessórios', estoque: 3 },  
];
```

1. **1.** Nomes em maiúsculas (**map**)
2. **2.** Produtos com preço acima de R\$ 40 (**filter**)
3. **3.** Valor total do estoque: soma de **preço * estoque** de cada produto (**reduce**)
4. **4.** Produto mais barato (**reduce**)
5. **5.** Quantos produtos há em cada categoria (um objeto contador)

Desafio: liste os produtos com estoque abaixo de 10.

Papelaria do Senac

A papelaria precisa de um sistema simples de controle de estoque. Vocês vão cadastrar os produtos e gerar um relatório da situação atual.

1. Repita (padrão sentinela): pergunte **nome**, **preço** e **quantidade** de cada produto; nome em branco encerra o cadastro
2. Guarde cada produto como um **objeto** dentro de um **array** (**estoque.push({ nome, preco, quantidade })**)

O relatório final deve mostrar

1. A lista de produtos, cada um classificado por **if/else if**: **ESGOTADO** (0), **BAIXO** (menos de 5) ou **OK**
2. Os itens que precisam de reposição (**filter**: quantidade < 5)
3. O **valor total** do estoque (soma de preço × quantidade de todos, com **reduce**)
4. O produto **mais caro** (**reduce**)
5. A lista de produtos ordenada do mais barato ao mais caro (**sort**, sem alterar a ordem original)

Inscrições do Evento Tech Senac

O Senac vai realizar um evento de tecnologia e precisa de um sistema de inscrições no terminal. Este projeto junta TUDO da aula: prompts, condicionais, laços e array de objetos.

1. Repita (padrão sentinela) enquanto houver **vagas** e o nome não vier em branco
2. Para cada inscrito, pergunte **nome (input)**, **categoria (select: Aluno do Senac, Egresso, Convidado externo)** e se **quer certificado (confirm)**
3. Guarde cada inscrito como um objeto num array

Regras e relatório final

Defina um limite de vagas (ex.: 5, para dar para testar) e pare o cadastro se ele for atingido.

1. Ao encerrar, mostre a **lista completa** de inscritos (**for...of** com desestruturação), formatada com **padEnd**
2. Mostre quantos inscritos há em **cada categoria** (objeto contador, como no exercício 4)
3. Mostre quantos **querem certificado** (**filter** + **.length**)
4. Responda: **há** algum convidado externo? (**some**) · **todos** pediram certificado? (**every**)

PROJETO INTEGRADOR

Esqueleto do projeto

```
import { input, select, confirm } from '@inquirer/prompts';

const LIMITE_DE_VAGAS = 5;
const inscritos = [];

while (true) {
  if (inscritos.length ≥ LIMITE_DE_VAGAS) {
    console.log('Evento lotado!');
    break;
  }
  const nome = await input({ message: 'Nome? (Enter encerra)' });
  if (nome.trim() === '') break;

  // TODO: perguntar categoria (select) e certificado (confirm)
  // TODO: inscritos.push({ nome, categoria, querCertificado })
}
// TODO: relatório final (abaixo do laço)
```

FECHAMENTO

Resumo da aula

- **Condicionais:** if, else if, else, switch, blocos e escopo
- **Repetições:** for, while, do...while, break/continue, o padrão sentinela
- **Arrays:** push/pop, map, filter, reduce, sort, desestruturação
- **Objetos:** propriedades, Object.entries, array de objetos
- **Tudo junto:** um sistema de inscrições completo

FECHAMENTO

Próximos passos

Funções

Declarar, parâmetros, retorno, reaproveitar código.

Organização

Dividir um programa grande em partes pequenas.

Ainda mais coleções

Arrays de arrays, objetos aninhados.

Para casa: refaça o mini-projeto trocando o array de inscritos por um menu com mais opções (listar, cancelar inscrição, buscar por nome).

FECHAMENTO

Para praticar em casa

Lista de questões

Questões sobre condicionais, laços e coleções, com respostas comentadas.

Gabarito comentado

Atividades, exercícios e as 4 atividades temáticas, resolvidos e explicados.

Como estudar

Tente antes, rode no Node e só então compare com o gabarito.

No fim da aula você recebe a **lista-de-questoes-aula2.md** e a pasta **gabarito/**.

OBRIGADO

Dúvidas e onde estudar mais

Recomendado: W3Schools

Páginas de if/else, switch, laços e arrays/objetos, com exemplos que você pode editar e executar. Conteúdo em inglês.

[Tutorial de JavaScript](#) · [Referência](#) · [Node.js](#)

MDN Web Docs (pt-BR): developer.mozilla.org/pt-BR

javascript.info: javascript.info

Node.js: nodejs.org/docs/latest/api

@inquirer/prompts: github.com/SBoudrias/Inquirer.js